QueueLess AI: PRD and One-Week Build Plan

Prepared for Peerace | NVIDIA x Nebius Hackathon (Best Apps and Agents) | Solo, 7 days

1. Overview

One-liner: QueueLess AI helps people prepare, choose, and sequence in-person service visits before leaving home, using verified requirements and transparent wait estimates to cut avoidable trips.

Positioning rule: The MVP promises preparation and planning, not guaranteed queue avoidance. No live queue data exists to rely on, so every claim the app makes must be labelled as verified, estimated, or unknown.

2. Problem and hypothesis

People who depend on in-person services (banks, NIN enrolment centres, licensing offices, hospital registration, university admin offices) lack reliable information about required documents, best visiting times, and how to order multiple errands. The result is repeat visits and lost days.

Hypothesis to validate: A large share of wasted time comes from missing documents and poor sequencing, not only from queue length. This is why the MVP focuses on checklists and planning.

3. Target user

Primary: Time-constrained students and young professionals in Nigerian university towns and cities who handle errands between classes or work.

Secondary (pitch only): Service operators and institutions wanting fewer unprepared visitors.

4. Goals and non-goals

Goals

- Produce a correct, task-specific document checklist for each supported service.
- Recommend a practical visit plan (what to do first, when to go, what to bring).

- Show every fact with its source, a timestamp, and a confidence label.
- Beat a plain-prompt baseline on a measurable evaluation.

Non-goals (explicitly out of scope)

- Real-time queue tracking or integrations with institutions.
- Booking appointments on the user's behalf.
- Covering more than two services or one city.
- Payments, accounts, or notifications.

5. MVP scope

Pick **two services** on Day 1, chosen by whether official requirement pages exist and are readable. Candidates: bank account opening (KYC), NIN enrolment or modification, driver's licence processes, or a campus administrative process (e.g., result or transcript requests). Rule: if you cannot find an official source page, drop the service.

Location scope: One place (Ile-Ife / Osun, or a campus), with a small hand-verified list of service locations.

6. Core user flow

1. User describes the errand(s) in plain language: "I need to open a bank account and fix my NIN details."
2. Agent identifies the services and asks at most 2 clarifying questions.
3. Agent retrieves the matching requirement entries from the curated corpus.
4. Agent outputs a structured plan: checklist, fees where sourced, suggested order, suggested time window, and a confidence label on each item.
5. After the visit, the user can log actual wait time and whether anything was missing (feeds the evaluation and the estimate feature).

7. Functional requirements

P0 (must ship)

- Free-text errand input.
- Service identification and clarifying questions.
- Checklist generation grounded only in the curated corpus.

- Every item shows: source link, date last verified, confidence (Verified / Estimated / Unknown).
- Refusal behaviour: if a fact is not in the corpus, the app says "not verified" and never guesses fees, hours, or waits.
- Multi-errand sequencing with a short rationale.
- Printable or shareable plan view (mobile-first).

P1 (if time allows)

- Visit report form: actual wait, missing documents, outcome.
- Wait estimate from collected reports, shown only when at least N reports exist, with a range and sample size.
- Evaluation dashboard page showing baseline vs QueueLess scores.

P2 (pitch only)

- Operator dashboard, appointment feeds, SMS or WhatsApp reminders, multi-city expansion.

8. AI design

- **Model:** A compatible NVIDIA Nemotron model through Nebius Token Factory (confirm the exact model ID and API format in the Nebius docs on Day 1).
- **Pattern:** Retrieval-grounded structured output. The model receives the user request plus only the matching corpus entries, and must return JSON that follows a fixed schema.
- **Steps:**
  1. *Intent extraction:* services, location, constraints, deadline.
  2. *Retrieval:* simple keyword or tag lookup over the corpus (a vector database is unnecessary at this size).
  3. *Plan generation:* checklist and order, with each item citing a corpus entry ID.
  4. *Validation:* code checks that every cited ID exists and that no fee, hour, or wait appears without a source. Failures trigger one retry, then an "unverified" label.
- **Guardrail:** The validator, not the prompt, enforces honesty. This is the technical story to show judges.

Plan schema (sketch)

```
{
  "errands": ["string"],
  "checklist": [{"item": "string", "source_id": "string", "confidence":
"verified|estimated|unknown"}],
  "order": [{"step": 1, "errand": "string", "reason": "string"}],
  "time_window": {"suggestion": "string", "basis": "string", "confidence":
"string"},
  "open_questions": ["string"]
}
```

9. Data model

- **ServiceEntry:** id, service name, location(s), required documents, fees (nullable), source URL, last verified date, notes.
- **VisitReport (P1):** id, service id, arrival time, wait minutes, missing item (optional), timestamp.
- Store as JSON files or a small Supabase/SQLite table. No user accounts needed.

10. Architecture

- **Frontend:** Next.js, mobile-first, one input page and one plan page.
- **Backend:** Next.js API routes calling Nebius Token Factory; schema validation with Zod.
- **Data:** Curated corpus in the repo (JSON), reports in a small hosted DB if P1 ships.
- **Hosting:** Vercel.

11. Evaluation plan

Build a test set of **12 to 15 scenarios** (single and multi-errand, with tricky cases such as a user who already has some documents).

Metric	How measured
Checklist completeness	Items matched against the official source, scored manually
Hallucination rate	Count of fees, hours, or claims with no source

Metric	How measured
Plan validity	Is the order logical and feasible? Yes/No per scenario
Unnecessary items	Extra documents not required

Compare against a **single-prompt baseline** using the same model and no corpus. Report results in a table in the demo and README. Report honestly even if the margin is small.

12. Success metrics

- At least 90% checklist completeness on the test set, and a clear improvement over the baseline.
- Zero unsourced fees or hours in QueueLess outputs.
- End-to-end flow works live in under 20 seconds.
- 5 or more real people try it and give feedback (ideally also 5 short interviews).

13. Risks and mitigations

Risk	Mitigation
Official info is outdated or hard to find	Show last-verified date; limit to two services; drop any without a source
No wait data	Keep waits out of the core promise; P1 only with real reports
Model invents details	Validator plus "unverified" fallback
Scope creep	Two services, one place, P0 first
API or model access issues	Test the Nebius call on Day 1; keep a fallback model call ready

14. Demo script (3 minutes)

1. **Hook (20s):** A real Nigerian queue story, ideally your own or one from an interview.

2. **Live run (60s):** Enter a two-errand request; show the clarifying question, then the plan with sources and confidence labels.
3. **Trust moment (40s):** Ask for something not in the corpus and show the app refusing to guess.
4. **Evidence (40s):** Show the baseline vs QueueLess table.
5. **Vision (20s):** Operator integrations and crowd-reported waits as the next step.

15. One-week build plan

Day 1: Foundations and data

- Confirm hackathon rules, deadline, and Nebius access; get the first Nemotron call working.
- Pick the two services; collect official source pages; write the first corpus entries.
- Start 3 to 5 quick interviews with students about their last bad queue experience.

Day 2: Corpus and schema

- Finish the corpus (JSON) with sources and dates for both services.
- Define the plan schema and Zod validators.
- Write the 12 to 15 test scenarios and the answer key now, before building more.

Day 3: Core agent

- Intent extraction, retrieval, and plan generation endpoints.
- Validator with retry and "unverified" fallback.
- Run the first pass on the test set.

Day 4: Frontend

- Input page, clarifying-question step, plan view with source and confidence badges.
- Mobile layout and a printable plan.

Day 5: Evaluation and hardening

- Run baseline vs QueueLess on all scenarios; record the table.
- Fix the worst failures; tighten prompts and validators.
- If time allows: visit report form (P1).

Day 6: Polish and proof

- Real-user tests with 5 or more people; collect quotes.

- Deploy, write the README (problem, architecture, evaluation, limits), and add an evaluation page.
- Record a backup demo video.

Day 7: Submit

- Rehearse the demo twice, finalise the pitch deck (Problem, Evidence and cost, Solution and USP), and submit with buffer before the deadline.
- Do not add features today.

16. Submission checklist

- Working deployed link and public repo.
- README with evaluation table and an honest limitations section.
- Demo video.
- Pitch that states clearly what is verified, what is estimated, and what is future work.
- Open-source notices and Nebius/NVIDIA usage clearly shown.